75.41 Algortimos y Programación II Curso 4
TP1

Memoria dinámica, punteros a funciones y otras cositas

10 de marzo de 2019

Fecha de presentación: 12/03/2019
Fecha de vencimiento: 19/03/2019

1. Introducción

El mundo conocido está constituido por cuatro continentes, Poniente, Essos, Sothoryos y Ulthos. Sólo los dos primeros son conocidos ampliamente, mientras los últimos permanecen en la incógnita.

El continente de **Poniente** reúne dos regiones mayores:

- Los Siete Reinos
  - **El Norte**, con capital en Invernalia.
  - **Tierras de los Ríos**, con capital en Harrenhal.
  - **El Valle de Arryn**, con capital en el Nido de Águilas.
  - **Islas del Hierro**, con capital en Pyke.
  - **Tierras del Oeste**, con capital en Roca Casterly.
  - **El Dominio**, con capital en Altojardín.
  - **Tierras de la Corona**, con capital en Desembarco del Rey.

    - **Tierras de la Tormenta**, con capital en Bastión de Tormentas.
    - **Dorne**, con capital en Lanza del Sol.

- Las tierras Más allá del Muro.

El continente de **Essos** reúne una vasta cantidad de regiones, algunas de las cuales están agrupadas en coaliciones de ciudades. Entre estos conglomerados están:

- Las Ciudades Libres
    - **Pentos**.
    - **Braavos**.
    - **Lys**.
    - **Qohor**.
    - **Norvos**.
    - **Myr**.
    - **Tyrosh**.
    - **Volantis**.
    - **Lorath**.
- La Bahía de los Esclavos
    - **Astapor**.
    - **Yunkai**.
    - **Meereen**.

Cada una de estas regiones está organizada, tienen nombre, muchas tienen un lema, y ninguna carece un ejército.

2. Objetivo

El presente trabajo práctico apunta a que el alumno se empiece a familiarizar con las herramientas a utilizar en la materia, el manejo de memoria dinámica y la utilización de punteros a funciones.

3. Enunciado

Se pide crear una biblioteca reino (.c y .h) que permita cargar y manipular todas las regiones del mundo conocido. Particularmente se espera que se implementen, dentro de la biblioteca, las siguientes funciones:

```c
/*
* Creará la estructura reino, reservando la memoria necesaria.
* Recibe el máximo de regiones que pueden ser cargadas,
* esta cantidad sirve para revervar la memoria necesaria
* para el vector de regiones.
* Devolverá la estructura, con el tope inicializado en 0.
* En caso de no poder crear el reino, devolverá NULL.
*/
reino_t* crear_reino(size_t maximo_regiones);

/*
* Destruirá la estructura creada para manipular el reino,
* liberando la memoria reservada para él y sus regiones.
* Devolverá 0 si pudo liberar la memoria o 1 si no pudo.
*/
int destruir_reino(reino_t* reino);

/*
* Agregará una región al vector de regiones del reino.
* Devolverá 0 si pudo agregarla o 1 si no pudo.
*/
int agregar_region(reino_t* reino, region_t region);

/*
* Ordenará el vector de regiones dentro del reino.
* La función de comparación de regiones vendrá como parámetro,
```

```
* esto permite ordenar el reino por cualquiera de los campos
* de la región.
*/
void ordenar_reino(reino_t* reino, int (*comparar_regiones)(region_t, region_t));

/*
* Listará por pantalla las regiones del reino.
*/
void listar_regiones(reino_t* reino);
```

Se cuenta además con 2 registros, uno que representa el reino y otro que representa las regiones del mundo conocido.

```
typedef struct region {
        char nombre[50];
        char lema[50];
        int ejercito;
} region_t;

typedef struct reino{
        region_t* regiones;
        int cantidad_regiones;
} reino_t;
```

La información de las regiones se encuentra en un archivo de texto **reino.txt** donde la primer línea solo contiene la cantidad máxima de regiones que puede contener el reino (será la utilizada para reservar la memoria para el vector de regiones).

Las demás líneas contienen el nombre de la región, el lema (si no tiene lema habrá un -) y la cantidad de soldados que contiene su ejército.

Los campos no poseen espacios para que puedan ser leídos con **fscanf** utilizando **%s**.

Ejemplo del archivo **reino.txt**

```
80
Stark;Se_acerca_el_invierno;86503
Lannister;Oyeme_rugir;1002345
...
```

Se deben crear **al menos** 2 funciones de comparación.

```
/*
 * Comparará 2 regiones según el nombre.
 * Devolverá:
 * -1 si el ejército de la region 1 es menor al de la región 2.
 * 0 si el ejército de la region 1 es igual al de la región 2.
 * 1 si el ejército de la region 1 es mayor al de la región 2.
 */
int comparar_por_ejercito(region_t region_1, region_t region_2){
        ...
}

/*
 * Comparará 2 regiones según la cantidad de ejército que tienen.
 * Devolverá:
 * -1 si nombre de la region 1 es menor al de la región 2.
 * 0 si el nombre de la region 1 es igual al de la región 2.
 * 1 si el nombre de la region 1 es mayor al de la región 2.
 */
int comparar_por_nombre(region_t region_1, region_t region_2){
        ...
}
```

4. Resultado esperado

Se espera que el trabajo creado sea compilado sin errores con la siguiente línea:

```
gcc *.c -Wall -Werror -Wconversion -std=c99 -o tp1
```

Luego que sea ejecutado, cargue y ordene las casas sin pérdida de memoria.

Para verificar el correcto funcionamiento, debe utilizarse un programa principal con el siguiente flujo:

```
int main(){

        /* Abrir el archivo */
        /* Crear reino */
```

```
        /* Agregar regiones */
        /* Ordenar reino por ejercito */
        /* Mostrar regiones */
        /* Ordenar reino por nombre */
        /* Mostrar regiones */
        return 0;
}
```

Y, ser corrido con **valgind**:

```
valgrind .\tp1
```

Obteniendo como resultado una corrida sin errores ni pérdida de memoria:

```
HEAP SUMMARY:
    in use at exit: 0 bytes in 0 blocks
  total heap usage: 3 allocs, 3 frees, 9,360 bytes allocated

All heap blocks were freed -- no leaks are possible

For counts of detected and suppressed errors, rerun with: -v
ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Referencias

https://hieloyfuego.fandom.com/wiki/Mundo_conocido